Site Reliability Engineering (SRE) Complete Study Notes

PHASE 1 — THE MENTAL MODEL

Topic 1: What Does SRE Stand For?

SRE stands for Site Reliability Engineering. Each word in the acronym carries precise meaning that defines the discipline.

S — Site

"Site" originated from "website" but today refers to any digital service delivered through the internet. This includes websites on desktop or mobile, smart TV applications, video games, smart home devices — essentially any software product a customer accesses over a network.

R — Reliability

"Reliability" means how stable, how fast, and how secure a service is. These three dimensions work together:

- **Stable** — the service does not present errors or failures
- **Fast** — the service responds quickly enough to meet customer expectations
- **Secure** — the service is trustworthy and protects customer data

E — Engineering / Engineer

"Engineering" refers to the application of scientific, software engineering methods to the problem of reliability. "Engineer" refers to the role — the person who performs SRE. Both meanings apply depending on context.

⚠ NOTE: SRE is not limited to websites. Any digital service delivered via the internet falls within the scope of SRE.

Topic 2: The Origin of SRE

SRE was born at Google. Ben Traynor Sloss, a senior software engineer at Google, was assigned responsibility for a production team in 2004. He realized that software engineering techniques and practices could be applied to the problem of operating large-scale systems — systems involving hundreds or thousands of servers.

From this realization came the famous phrase:

```
"SRE is what happens when a software engineer is asked to do operations."
```

Over the next ten years, SRE was refined and spread internally at Google as the gold standard for operating large-scale systems. In 2014, the first SREConf was held, promoted by USENIX (the Advanced Computer Systems Association). At that conference, Ben Traynor Sloss presented his keynote "The Keys to SRE", listing practices that eventually became the seven SRE practices known today. This presentation effectively launched SRE as a recognized IT field.

SRE is now widely sought after in job postings and adopted by Fortune 500 companies globally.

⚠ NOTE: The 2014 SREConf keynote by Ben Traynor Sloss is available on USENIX.org and is considered the founding document of SRE as a discipline.

Topic 3: It Is All About Customer Expectations

Reliability is ultimately a feeling, not just a measurement. Customers form expectations about services they use. When those expectations are consistently met with minimal errors, the service feels reliable. When they are not, the customer feels one of several negative emotions:

- **Fear** — e.g., a structurally suspect bridge that nobody wants to cross
- **Insecurity** — e.g., an airline where safety is uncertain
- **Frustration** — e.g., a car that breaks down on the road
- **Anger** — e.g., an unresponsive customer support system

When expectations are consistently met, customers feel happy. A happy customer is the best advertisement — they return, they recommend, and they tolerate minor imperfections.

Different services carry different expectations. A Jeep is expected to handle rough terrain. A city car is expected to be easy to park. A calculator needs simple arithmetic. A stock market platform needs high security, high speed, live data updates. Meeting the wrong expectations for a given service frustrates everyone.

⚠ NOTE: SRE is fundamentally about meeting customer expectations regarding reliability so that customers remain happy. Understanding what those expectations are is the starting point of all SRE work.

Topic 4: How We Measure Reliability — SLI

Feelings cannot be measured directly, but the facts that produce those feelings can. SRE measures reliability by tracking good and bad events and calculating the ratio between them.

Examples of event-based measurements:

- **Flat tire rate** — bad event = flat tire per 10,000 miles driven
- **Success Transaction Rate** — good event = payment processed successfully vs. payment failed
- **Page Load Rate** — good event = page loaded in under 500ms; bad event = page loaded in over 500ms

The ratio of good to bad events tells us whether we are achieving the customer's reliability expectations. These indicators are called Service Level Indicators (SLIs).

⚠ NOTE: SLI = Service Level Indicator. It is the measurement. It answers the question: "Are we achieving our reliability targets right now?"

Topic 5: The Five-Nines — Highest Reliability Level

Five-Nines (99.999%) is the gold standard of availability and represents near-perfect reliability. The number of nines corresponds to downtime allowed per month:

Nines	Percentage	Monthly Downtime Allowed
Five-Nines	99.999%	~26 seconds
Four-Nines	99.99%	~4 min 22 sec
Three-Nines	99.9%	~43 minutes
Two-Nines	99%	~7 hours
One-Nine	90%	~73 hours

Five-Nines sounds desirable, but it comes with critical caveats. It is very expensive to achieve. Achieving it means leaving almost no room for new features or innovation — which is exactly what customers also want. Pursuing Five-Nines can produce a state of inertia where nothing new is delivered.

⚠ NOTE: More reliability than customers actually need is a waste of money. The goal is not Five-Nines — the goal is the right reliability level for the business and its customers. Availability is also only one dimension of reliability; customers also care about speed and behaviour.

Topic 6: SLA — The Minimum Reliability Level

SLA stands for Service Level Agreement. It is a contractual commitment made between a service provider and its customers or regulators. An SLA defines the minimum acceptable level of service.

If an organization fails to meet its SLA, it must compensate its customers. SLAs are established by:

- **Legal obligation** — government regulation mandating a minimum service level
- **Contractual obligation** — explicit agreements with enterprise or government customers

The critical mistake companies make is treating the SLA as their goal. Using the SLA as a target means operating right at the edge of acceptability, leaving no buffer for failures and no margin for innovation.

⚠ NOTE: SLA is NOT the goal. It is the floor — the minimum you must not fall below. Treating SLA as a target means you are already nearly failing your customers.

Topic 7: SLO — The Desired Reliability Level

SLO stands for Service Level Objective. Unlike SLAs (which are external obligations), SLOs are internal goals that the team sets for itself. They define the desired reliability level — more ambitious than the SLA but realistic enough to be achievable within budget.

Characteristics of a well-defined SLO:

- **Realistic** — achievable given the team's resources and the budget available
- **More ambitious than the SLA** — gives a buffer so that normal variance doesn't breach the contract
- **Customer-driven** — based on actual customer expectations, not arbitrary numbers
- **Gives margin for innovation** — must not require 100% perfection; leaves room to ship new features

A service rated 4.5/5 stars does not achieve Five-Nines. In availability terms, 4.5/5 is roughly equivalent to One-Nine (90%). Yet that app can generate billions in revenue because it innovates constantly and still meets expectations. That is the power of a well-set SLO.

⚠ NOTE: The most critical property of an SLO: it MUST give margin for innovation. An SLO that demands too much reliability kills the team's ability to ship new features.

Topic 8: Error Budget — Balancing Reliability and Innovation

The Error Budget is derived directly from the SLO. It is the allowed amount of unreliability for a given time period. As long as the error budget has not been exhausted, the team is free to release new features, deploy changes, and innovate. Once the budget is spent, all new releases are frozen until the budget recovers.

An analogy: while you are driving within the speed limit, you may accelerate freely. The moment you exceed the limit, you brake — you stop deploying until reliability recovers.

Why the Error Budget is transformational for organizations:

- **Shared ownership** — Bad reliability previously impacted Ops only — more tickets, nights awake. Now it stops Dev from releasing. Everyone is in the same boat.

- **Business awareness** — Business also cannot ship new features when the budget is spent. Reliability becomes a business concern, not just an Ops concern.
- **Rational conversations** — Teams can now evaluate whether their SLOs are correctly set: Can we stop services at weekends? Does this SLO actually reflect customer expectations?

⚠ NOTE: Error Budget is what makes safely innovating possible. It creates the equilibrium between reliability and innovation by making reliability consequences visible to everyone — Dev, Ops, and business.

Comparison: SLI vs SLA vs SLO vs Error Budget

Concept	What It Is	Who Sets It	Key Purpose
SLI	Measurement — good vs bad events ratio	SRE team	Tells you if you are meeting targets right now
SLA	Contractual minimum reliability commitment	Legal/regulators/contracts	The floor — never go below this
SLO	Internal desired reliability objective	SRE + product team	The real goal — above SLA, leaves room to innovate
Error Budget	Remaining allowed unreliability for the period	Derived from SLO	The license to release and innovate safely

PHASE 2 — THE 7 SRE PRINCIPLES

Principles are the foundational pillars of SRE. They are immutable — they should be the same across all organizations regardless of size, industry, or structure. Principles provide the philosophical basis from which every SRE decision is made.

⚠ NOTE: Principles are universal. Practices are shaped to fit the organization. Never confuse the two.

Principle 1: Embrace Risk

Embrace Risk means accepting that 100% reliability is impossible and that chasing it is counterproductive. Every service running on the internet depends on other services that can fail. There is no way to eliminate risk entirely.

More importantly, reliability costs money. Every additional "nine" of availability requires exponentially more investment in infrastructure, redundancy, and engineering time. Money spent on over-engineering reliability is money not spent on new features.

The principle teaches that innovation requires risk. Delivering new features to customers means deploying changes — and deployments carry risk. Embracing risk means accepting this reality and managing it intelligently rather than trying to eliminate it.

⚠ NOTE: "The perfect is the enemy of the good." Chasing perfection in reliability starves the business of innovation. Embrace Risk means finding the right balance.

Interview Takeaway

If asked about Embrace Risk: explain that 100% reliability is impossible, that reliability costs money, and that the SRE goal is a balance — enough reliability to keep customers happy while preserving budget and capacity for innovation.

Principle 2: Service Level Objectives

This principle mandates that organizations define explicit, measurable reliability goals — the SLOs. Measurement without goals is meaningless. The SLO principle forces teams to translate customer expectations into concrete targets.

Key requirements of this principle:

- **Define SLOs** — set internal targets above the SLA minimum
- **Multiple SLOs** — one service may have multiple SLOs — one for latency, one for error rate, one for availability
- **Realistic** — SLOs must be achievable given budget; 100% is never an appropriate SLO
- **Customer-informed** — SLOs must be grounded in what customers actually expect

⚠ NOTE: SLOs must give room for innovation. An SLO that leaves no error budget kills the team's ability to deploy.

Principle 3: Eliminate Toil

Toil refers to repetitive, manual, low-value work that has to be done regularly but produces no lasting improvement or business value. Like doing laundry — it needs doing, but doing it more does not make you better at it and it never gets "done" permanently.

Why toil is dangerous in engineering:

- **Does not scale** — more work = more people required; cannot grow without growing headcount
- **No long-term value** — the same task will need to be done again tomorrow
- **Wastes engineering talent** — toil prevents engineers from working on problems that actually improve the system

The message of this principle is: invest in people by freeing them from toil. Remove manual, repetitive work through automation so that engineers can focus on meaningful engineering.

⚠ NOTE: Toil is not just "work I dislike." It is specifically work that does not scale, produces no lasting value, and grows proportionally with system complexity rather than being automated away.

Interview Takeaway

Toil = manual, repetitive, scaleless work. Eliminating it means building automations so that growing the system does not require proportionally growing the team.

Principle 4: Monitoring

Monitoring is both a principle and a practice (it appears in both the 7 principles and the 7 practices). As a principle, it establishes that observability of services is non-negotiable — you cannot manage what you cannot see. Monitoring is the foundation upon which all other SRE work depends.

⚠ NOTE: Monitoring is the mother of all SRE practices. Without monitoring, incident response, postmortems, capacity planning, and SLI measurement are all impossible.

Principle 5: Automation

Automation means delegating repetitive, mechanical tasks to machines. Machines excel at repetitive work — they do not tire, do not make attention-based mistakes, and can execute tasks faster and more consistently than humans.

Benefits of automation:

- **Reduces human error** — eliminates mistakes caused by fatigue or inattention
- **Increases speed** — machines execute faster than humans

- **Enables scale** — the amount of work can grow without the team growing proportionally

The core message of this principle is that automation is the mechanism by which SRE teams scale. A team of five engineers can operate a system of any size if the system is sufficiently automated.

⚠ NOTE: Automation is how you scale. Without it, every increase in system complexity or workload requires additional headcount. With it, the system grows without the team growing.

Principle 6: Release Engineering

Release Engineering is the practice of having explicit control over everything that is delivered and the entire process by which it is delivered. Think of it like an assembly line: every step from raw materials to finished product is defined, measurable, and quality-controlled.

Release Engineering matters because the most common time for systems to fail is immediately after a change is deployed to production. Bugs introduced during deployment, untested code reaching customers, and rollback failures are the leading causes of outages.

Release Engineering addresses this by:

- **Defining the pipeline** — every step from concept to production is explicitly planned
- **Quality gateways** — gates between stages catch defects before they reach customers
- **Rollback planning** — if something goes wrong, the path back to the previous stable state is pre-defined

⚠ NOTE: The launch moment is the most dangerous moment for reliability. Release Engineering exists to make deployments as safe and controlled as possible.

Principle 7: Simplicity

Simplicity means focusing on what is essential — removing everything that does not serve the customer or the mission. Complexity is the enemy of reliability because complex systems are harder to understand, harder to debug, and harder to hand over to new team members.

How simplicity serves SRE:

- **Noise reduction** — eliminating irrelevant signals so the important ones are visible
- **Cost reduction** — using only the resources actually needed
- **Knowledge sharing** — simple systems can be understood and operated by more people
- **User experience** — both internal tools and external products should be as simple as they can be

An important caveat: simplicity is not easy. It requires discipline, effort, and regular refactoring. Complexity accumulates naturally over time; simplicity must be actively maintained.

⚠ NOTE: You cannot say something is reliable if you cannot understand it. Simplicity is a prerequisite for reliability, not a nice-to-have.

Summary: The 7 SRE Principles

Principle	Core Message	Interview Hook
1. Embrace Risk	100% reliability is impossible and costly	Balance reliability with innovation budget
2. Service Level Objectives	Define measurable reliability goals	SLO must leave room for innovation
3. Eliminate Toil	Remove repetitive, scaleless manual work	Invest in people by automating toil

4. Monitoring	You cannot manage what you cannot see	Foundation of all other SRE work
5. Automation	Machines handle repetitive work at scale	Automation is how teams scale without headcount
6. Release Engineering	Control the entire delivery pipeline	Deployments are the riskiest moment — engineer them
7. Simplicity	Complexity kills reliability	Simplicity requires discipline and active effort

PHASE 3 — THE 7 SRE PRACTICES & THE RELIABILITY PYRAMID

Practices are where theory becomes reality. Unlike principles, which are universal, practices are shaped to fit the organization — the size of the team, the tools available, the nature of the services. Practices are how you get your hands dirty.

The 7 SRE Practices form a pyramid. Each practice builds on the one below it. You cannot do Incident Response well without Monitoring. You cannot do Root Cause Analysis without Incident Response. The order matters.

The Reliability Pyramid (bottom to top)

- **Level 1 (base)** — Monitoring — the foundation everything else depends on
- **Level 2** — Incident Response — built on monitoring alerts
- **Level 3** — Root Cause Analysis & Postmortems — built on incident records
- **Level 4** — Testing & Release Engineering — prevention before problems occur
- **Level 5** — Capacity Planning — optimization when systems are stable
- **Level 6** — Development — internal tooling built on all practices below
- **Level 7 (top)** — User Experience — the goal; everything else serves this

⚠ NOTE: The lower in the pyramid, the more frequently you perform that practice. You monitor constantly. You run incidents reactively. Capacity planning happens once or twice a year.

Practice 1: Monitoring

Monitoring is the practice of collecting, analyzing, and processing relevant events and facts from services. Data arrives either by pulling from logs or by services pushing their own metrics. Raw events are aggregated into metrics, displayed in dashboards, and compared against expected behavior so that the team is automatically notified when something goes wrong.

Reasons to monitor:

- Know what is happening at all times
- Detect and respond to incidents quickly
- Identify the root cause of problems after the fact
- Find bottlenecks and improve performance
- Detect trends — usage growing, latency trending up
- Identify waste — resources provisioned but not used
- Measure the impact of a change after deployment
- Understand user behavior and usage patterns

Monitoring tells the team when a service is healthy, when it is saturated, when it is idle, and when it is failing.

⚠ NOTE: Monitoring is the mother of all SRE practices. Every other practice — incident response, RCA, capacity planning, SLI measurement — depends entirely on monitoring being in place first.

Interview Takeaway

If asked about monitoring in SRE: it is both principle and practice, it is the foundation of the reliability pyramid, and all other SRE work depends on it. A service you cannot monitor is a service you cannot reliably operate.

Practice 2: Incident Response

Incident Response is the practice of being prepared before incidents happen so that when they do occur, the team can respond rapidly, minimize customer impact, and restore service as fast as possible.

Core components of a mature Incident Response practice:

Fire-Drill Simulations

Like firefighters practicing fire drills before a real fire, SRE teams simulate incidents in advance. This reveals how systems behave under failure, who needs to be involved, and what the customer impact will be.

On-Call

On-Call is the allocation of specific engineers to monitor the system and respond immediately if an alert fires. Monitoring automatically notifies the on-call engineer based on pre-defined alerting rules.

Notification Plan

The Notification Plan defines who is alerted and when. Good notification design avoids alert noise — false alarms that wake engineers without any real customer impact. Alerts should only fire when someone needs to act.

Logistic Plan

The Logistic Plan covers the practical side of incident response: how engineers get to the office if required, what equipment and access they need, what emergency tools are available. Think of the fire truck, the hose, and the ladder.

Communication Plan

The Communication Plan answers: who needs to know something is happening? Managers? Other on-call engineers? Customers? Having a communication plan ensures the right people are informed at the right time without creating panic or noise.

⚠ NOTE: The primary goal of Incident Response is to REDUCE THE DAMAGE — minimize customer impact and restore service to normal as fast as possible. Investigation of cause comes afterwards.

Interview Takeaway

Incident Response = preparation before incidents. On-Call + Notification Plan + Logistic Plan + Communication Plan. The goal is damage reduction and fast recovery, not immediate root cause analysis.

Practice 3: Root Cause Analysis and Postmortems

Root Cause Analysis (RCA) is the investigation of what caused an incident. It is directly linked to Incident Response — once the incident is contained and service is restored, the RCA begins.

Blast Radius

The first priority during an incident is to reduce the blast radius — minimize the number of customers affected and the severity of the impact. This may mean applying a quick fix (like expanding a full database) without yet knowing the root cause. Understanding comes after service is restored.

When to Perform RCA

RCA should ideally be conducted after the incident is resolved, with calm — preferably the following day, after all relevant data has been collected. Rushing an RCA during a live incident produces incomplete analysis.

The Postmortem Document

A Postmortem is a formal document that records all relevant facts about the incident in timeline order. It includes:

- The exact time the incident was first detected
- The exact time customers were notified
- Every action taken and its timestamp
- The exact time service was restored
- The root cause identified by the investigation
- Action items to prevent recurrence

Blameless Postmortems

A blameless postmortem focuses on system failures and process gaps, not individual blame. The goal is learning, not punishment. When postmortems are blameless, engineers are more honest about what happened, leading to better analysis and better fixes.

Why Postmortems Are Valuable

- **Learning from failure** — makes the same incident less likely to recur
- **Faster future resolution** — when the same incident recurs, engineers already know the fix
- **Organizational memory** — new engineers can read past postmortems to understand the system's history
- **Cross-company sharing** — sharing postmortems between companies helps the industry learn collectively

⚠ NOTE: The postmortem practice is about continuous improvement. Its purpose is to ensure the same failure does not repeat, not to assign blame.

Interview Takeaway

RCA = investigation of cause. Postmortem = formal document with timeline. Blameless = focus on system, not people. Key message: postmortems are for learning and preventing recurrence.

Practice 4: Testing and Release Engineering

Testing — Prevention

Tests are quality gateways. They prevent bugs and bad code from reaching customers by catching defects in safe environments before deployment. While Incident Response reacts to fires and Postmortems analyze what went wrong, Testing exists to prevent the fire from starting.

Code Coverage is a metric that shows what percentage of the codebase is exercised by tests, helping identify untested areas.

Test Types

Test Type	Purpose	Scope
Unit	Test individual functions or components	Smallest — isolated component
Integration	Test how components work together	Multiple components interacting
System	Test the full system as a whole	End-to-end
Smoke	Verify system starts up with minimal features working	Basic health check after deployment
Regression	Ensure new changes do not break existing functionality	Full feature set
Load / Performance	Verify system behaves correctly under high load	Simulated peak traffic

Release Engineering — Efficiency

Release Engineering defines the entire pipeline from the moment a feature is conceived to the moment it reaches customers. It is not just about deployment — it is about every step in between, including all the gates where quality can be verified.

Key elements of Release Engineering:

- **Environments** — Development → Test → Staging → Production — each stage gates quality before promotion
- **Pipeline automation** — manual steps are replaced with automated processes (CI/CD)
- **Continuous Delivery/Deployment** — automated, frequent, low-risk release cadence
- **Configuration Management** — non-code changes that affect the production environment are tracked and controlled
- **Rollback planning** — when something goes wrong, the path back to the last stable state is pre-planned

⚠ NOTE: Release Engineering is about both prevention and efficiency — preventing bad code from reaching customers, and making the path from idea to customer as fast and safe as possible.

Interview Takeaway

Testing = prevention, quality gateways. Release Engineering = the planned, automated pipeline from conception to production, with rollback capability. Together they answer: "How quickly can we go from idea to customer delivery, without breaking things?"

Practice 5: Capacity Planning

Capacity Planning is an optimization practice. It is done when systems are stable — not during incidents. Typically performed once or twice per year. The goal is to ensure you have exactly the right amount of infrastructure — not too much (waste) and not too little (outage risk).

Capacity Planning is based on monitoring history. The historical data from monitoring shows usage patterns, peak periods, and trends over time.

Why Capacity Planning Matters

- **Underutilized resources** — wasted money — servers running at 5% capacity are still paid for
- **Overloaded resources** — directly correlate with increased errors and latency — reliability impact

Scaling Terminology

Direction	Meaning	When to Use
Scale Out	Add more instances (horizontal)	Handle more parallel requests
Scale In	Remove excess instances (horizontal)	Reduce waste during low traffic
Scale Up	Replace with more powerful resources (vertical)	Single process needs more CPU/RAM
Scale Down	Replace with less powerful resources (vertical)	Current resources are oversized

For longer-horizon planning (days, weeks, years), Capacity Planning means anticipating predictable peaks: Black Friday traffic spikes, end-of-quarter reporting loads, new product launches. This is planned months in advance, not reactively.

⚠ NOTE: Capacity Planning allows you to do more with less. It reduces both waste (underutilization) and reliability risk (overload). Given cloud pricing, Capacity Planning directly impacts the monthly infrastructure bill.

Interview Takeaway

Capacity Planning = right-sizing your infrastructure. Done periodically based on monitoring history. Two failure modes: over-provision (waste) and under-provision (outage). In Azure, this maps to scaling rules on VMSS, AKS node pools, and App Service Plans.

Practice 6: Development

SRE is a specialization within software engineering. SREs are software engineers and sysadmins first — they bring the same practices and standards as other engineers, but they focus on internal products (tools, automations, pipelines) rather than customer-facing products.

The blacksmith analogy: a blacksmith makes tools both for their own use and for colleagues. SREs are the blacksmiths of the engineering organization — they build the tools that make everyone else's reliability work possible.

The 50/50 Split

A Site Reliability Engineer splits their time roughly evenly:

50% — Development Tasks	50% — Operations Tasks
Write automation scripts	Monitor infrastructure and applications
Build internal tooling	Respond to on-call incidents
Define and maintain CI/CD pipelines	Conduct capacity planning
Design software alongside product engineers	Write postmortems and runbooks

⚠ NOTE: If an SRE is spending more than 50% of their time on operations and toil, something is wrong. The development half is essential — it is how the team builds automation to reduce future toil.

Interview Takeaway

Development practice = dedicated engineering effort to build internal tools and automation. It is what separates SRE from pure operations. Without it, the team cannot eliminate toil or scale.

Practice 7: User Experience (UX)

User Experience sits at the top of the reliability pyramid. It is both a practice and the goal of all SRE work. Everything below it on the pyramid exists to serve it.

Two Dimensions of User Experience

Dimension	Focus	How SRE Serves It
Internal Users (engineers, ops)	Simplicity and productivity	Build tools that abstract complexity; automate manual steps; make the system easy to operate
External Users (customers)	Balance of innovation and reliability	Deliver new features frequently while maintaining stability and speed

The keyword for this practice is Goal. Every time you perform any other SRE practice — setting up monitoring, responding to incidents, planning capacity — you must keep both internal and external user experience clearly in mind as the reason for doing it.

⚠ NOTE: User Experience is not just about customer-facing UI. It also means how easy it is for internal teams to operate the system. A complex, opaque system with poor tooling is a reliability risk because it makes every other SRE practice harder.

Summary: The 7 SRE Practices

Practice	Pyramid Level	Key Message
Monitoring	1 — Foundation	Mother of all practices; everything depends on it
Incident Response	2	Reduce damage; be prepared before incidents happen
RCA & Postmortems	3	Learn from failure; prevent recurrence; blameless
Testing & Release Eng.	4	Prevention and efficient delivery pipeline
Capacity Planning	5	Do more with less; right-size infrastructure
Development	6	Build internal tools; 50/50 Dev/Ops split
User Experience	7 — Goal	The reason for everything else; internal and external

PHASE 4 — THE SRE ROLE

Topic 1: What Is a Site Reliability Engineer?

A Site Reliability Engineer is a software engineer or sysadmin who has specialized in building and maintaining highly reliable large-scale systems. The definition has three parts:

- Software engineer or sysadmin — they start from one of these two backgrounds
- Experienced — they need real prior experience in their background role, not a beginner
- Specialized in reliability — they apply their engineering skills to the problem of making systems reliable at scale

Crucially, there is no dedicated college course or certification to become an SRE. This makes SREs genuinely scarce in the market. You become an SRE by being an experienced engineer who has learned the principles, practices, and tools of SRE.

⚠ NOTE: SREs do not need to be the most senior engineer in the room. They need experience in software engineering or sysadmin roles, plus deep knowledge of SRE principles and practices.

Topic 2: What Does a Site Reliability Engineer Do?

SRE work divides cleanly into development tasks and operations tasks, with roughly equal time allocated to each.

Development Tasks (50%)

- **Application performance monitoring** — monitoring focused on application behavior, not just hardware
- **Pipeline authoring** — writing and maintaining CI/CD pipelines
- **Internal tooling** — coding automations and internal tools to eliminate toil
- **Design consultation** — helping product engineers design software that is simple to operate, fast, and secure

Operations Tasks (50%)

- **Infrastructure monitoring** — observing VMs, containers, cloud services — hardware-level concerns
- **On-call / Incident Response** — being available and responding when alerts fire
- **Capacity Planning** — right-sizing infrastructure based on monitoring data
- **Postmortems** — writing formal documents after incidents to learn and prevent recurrence
- **Runbooks** — documenting operating procedures for routine and emergency operations

At a higher level, the SRE is also responsible for ensuring the SRE principles and practices are being followed — acting as an internal consultant or specialist on reliability.

⚠ NOTE: SRE is the practical implementation of Dev+Ops. Not DevOps in the cultural sense, but an engineer who literally does both development and operations work with a specialization in reliability.

Topic 3: Who Can Be an SRE?

The ideal SRE profile shares common traits whether the candidate comes from a software engineering or sysadmin background:

Traits from Software Engineering Background

- **Likes to automate their job away** — naturally seeks to eliminate manual work through code
- **Knows application monitoring** — understands how to instrument code, read logs, measure performance
- **Can code** — writes scripts and programs in well-known languages
- **Understands the hardware-software-people triad** — reliability is not purely a technical problem; it is also a communication and organizational problem

Traits from Sysadmin Background

- **Likes to automate their job away** — gets bored doing the same thing twice; instinctively scripts it
- **Knows infrastructure monitoring** — cloud monitoring, server metrics, network behavior
- **Can script** — at minimum Bash; ideally Python or Node.js
- **Understands the hardware-software-people triad** — knows reliability spans hardware, software, and organizational communication

Both Must

- Know SRE principles and practices thoroughly
- Understand the reliability pyramid and why the order matters

⚠ NOTE: SREs do not need to be superhero engineers. They need the right profile: automation mindset, monitoring knowledge, coding ability, and deep understanding of SRE principles and practices.

PHASE 5 — ADOPTING SRE

Topic 1: The First Steps

When an organization decides to adopt SRE, the first step is always monitoring. Before SLAs, SLIs, and SLOs can be defined, the organization must have visibility into its services. Without monitoring, none of the other SRE work is possible.

The adoption sequence:

```
Step 1: Set up Monitoring
  → Services must communicate their health and behavior through metrics

Step 2: Identify SLAs
  → Involve: legal, business, security teams
  → SLA = the minimum — not the goal

Step 3: Define SLIs
  → Identify the good/bad events that measure reliability against the SLAs

Step 4: Set SLOs
  → Involve: UX specialists, product owners, architects
  → SLO = more ambitious than SLA, but realistic
  → Must leave room for innovation (error budget)
```

SRE adoption changes the release process. Regardless of how releases were managed before — CAB meetings, self-assessments — SRE adds an error budget gate. Before any release is allowed through, the error budget must have capacity remaining.

⚠ NOTE: Communication is vital. SRE adoption involves developers, operations, business, legal, security, product, and UX. All must be on the same page before and during adoption. Without organizational alignment, SRE will fail even if the technical pieces are in place.

Topic 2: The Groundwork (Pre-SRE Adoption)

Before hiring the first SRE or forming the first SRE team, there is valuable preparatory work any organization can do. These steps lay the foundation and can deliver significant reliability benefits on their own:

- **Create a communication plan** — start spreading awareness of what SRE is and why the organization is adopting it
- **Spread SRE principles** — educate teams on Embrace Risk, Eliminate Toil, Automation, etc.
- **Define initial SLAs, SLIs, SLOs** — even rough versions are better than none
- **Promote blameless postmortems** — begin the cultural shift toward learning from failure
- **Improve incident response** — create on-call rotations, notification plans, logistic plans
- **Foster simplicity and automation** — begin eliminating toil before the SRE team exists

⚠ NOTE: The groundwork phase can bring real improvements before a single SRE is hired. Spreading the principles alone changes the culture — and culture is 80% of SRE adoption success.

Topic 3: SRE Team Formats

Every organization is different. The right SRE team structure depends on the company's size, maturity, service diversity, and existing team structure. There is no single right answer. Each model has tradeoffs.

Format	Description	Best For	Risk/Drawback
Dedicated SRE Team (Consultants)	A central SRE team that advises and trains other teams	Companies open to SRE; single SRE team can meet all needs; initial SRE implementations	SRE team may be seen as external; teams may ignore guidance if not embedded
Ops Team IS the SRE	Existing Ops team adopts SRE practices and becomes the SRE team	Initial SRE adoption; similar services; Ops open to SRE principles	Devs and Ops remain separated; not true collaboration; some Ops may resist the change
SREs Embedded in Dev Teams	SRE members work alongside product development teams	Critical services needing high reliability; cross-functional teams; no dedicated Ops staff	Requires Dev teams to embrace SRE; SREs may become isolated from other SREs
Dedicated SRE + SREs in Teams (Hybrid)	Both a central SRE team and SREs embedded in dev and ops teams	Large companies; mature SRE organizations; services of varying criticality	Requires significant investment; needs rotation management and SRE career ladder

⚠ NOTE: Team formats are not static. An organization may start with one format and evolve to another as SRE maturity grows. Starting small and evolving is expected and recommended.

Topic 4: SRE Guilds

SRE Guilds are not a team format — they are a recommendation for preventing SRE members from becoming isolated, especially when SREs are embedded in different product teams.

A Guild is a regular meetup of all SRE members across the organization — regardless of which team they are embedded in. Guilds serve to:

- **Share experiences** — both good lessons and post-incident learnings
- **Align on standards** — agree on shared tooling, practices, and definitions
- **Evaluate tools** — collectively assess new tools being considered for adoption
- **Build community** — SREs working in different teams still identify as a community

⚠ NOTE: Guilds are most important when SREs are scattered across different development teams. Without a guild, embedded SREs can drift from each other in practices and lose the collective knowledge of the SRE community.

Topic 5: SRE Adoption Challenges

SRE adoption is not purely a technical challenge. The human and organizational dimensions are harder than the tooling. Common challenges:

Challenge	Description
Resistance to change	Teams and individuals are comfortable with how they work today
Scarcity of SRE talent	SREs are rare — no dedicated education path; companies compete for a small talent pool
Feature team resistance	Teams focused on delivery see SRE as a slowdown or constraint
Management resistance	IT management may resist the error budget release gate and loss of control over release timing
Lack of training	Teams do not understand SRE principles and practices well enough to apply them
Misconceptions	SRE is often confused with DevOps, ITIL, or traditional operations

Organizations with an established agile and DevOps culture adopt SRE more readily, because SRE is built on the same foundations — agile principles underlie DevOps, and DevOps principles underlie SRE.

Overcoming these challenges requires:

- **Senior management sponsorship** — SRE must be visibly championed from the top
- **Clear communication** — everyone must understand what SRE is, why it is being adopted, and what it means for their work
- **Evolutionary adoption** — start small, show results, expand; do not try to transform everything at once

⚠ NOTE: Like DevOps, the success of SRE adoption is more about culture, communication, and organizational change than about tools and processes. Results should begin to appear within months of starting adoption.

PHASE 6 — INTERVIEW FAVOURITES & REFERENCE MATERIAL

Interview Question 1: Explain SLI, SLA, SLO, and Error Budget

This is the single most common SRE interview topic. Here is a clear, structured answer:

```
SLI  (Service Level Indicator)
  What it is: A measurement — the ratio of good events to total events.
  Example: "Pages loaded in under 500ms" / "Total page requests" = 95%
  Key message: SLI is how you observe reliability. It answers "are we there yet?"

SLA  (Service Level Agreement)
  What it is: A contractual minimum. If you breach it, you pay compensation.
  Example: "We guarantee 99.9% availability per month or provide credits."
  Key message: SLA is the FLOOR, not the goal. Never treat it as a target.

SLO  (Service Level Objective)
  What it is: Your internal target. More ambitious than SLA. Self-imposed.
  Example: "We aim for 99.95% availability" (while SLA requires 99.9%)
  Key message: SLO must be realistic AND must leave room for innovation.

Error Budget
  What it is: The allowed failure margin derived from the SLO.
  Example: 99.95% SLO = 0.05% error budget per month = ~21 minutes of downtime
  When full: releases stop. When not exhausted: releases proceed freely.
  Key message: Error Budget is what enables safe innovation.
```

Interview Question 2: Principles vs Practices — What is the difference?

Principles are foundational, universal, and immutable — they apply the same way to every SRE implementation regardless of organization size or industry. Practices are shaped by the organization: the team size, the tools chosen, the services being operated.

Dimension	Principles	Practices
Universality	Same in all organizations	Shaped by organization context
Nature	Philosophical foundations — the WHY	Practical methods — the HOW
Mutability	Immutable — do not change across implementations	Adaptable — tailored to team and tooling
Count	7 principles	7 practices
Example	Embrace Risk (always applies)	Incident Response (process varies by org)

Interview Question 3: What is the Reliability Pyramid?

The Reliability Pyramid shows the order in which SRE practices build on each other. Lower practices must be in place before higher ones are meaningful.

```
┌──────────────────────────────┐
│      7. User Experience      │  ← The goal
├──────────────────────────────┤
│      6. Development          │
├──────────────────────────────┤
│      5. Capacity Planning    │
├──────────────────────────────┤
│   4. Testing & Release Eng.  │
├──────────────────────────────┤
│   3. RCA & Postmortems       │
├──────────────────────────────┤
│   2. Incident Response       │
├──────────────────────────────┤
│   1. Monitoring              │  ← The foundation
└──────────────────────────────┘

Lower = done more frequently. Higher = built on what is below.
```

Interview Question 4: Why is Five-Nines not always the goal?

Five-Nines (99.999%) sounds ideal but is often counterproductive:

- It is extremely expensive — the infrastructure redundancy and engineering effort required is disproportionate for most services
- It consumes the entire error budget — leaving no room for deployments, innovation, or new features
- Customers often cannot perceive the difference between 99.99% and 99.999% reliability
- Most services depend on external dependencies that cannot themselves guarantee Five-Nines

A well-known example: a major consumer app rated 4.5/5 stars (equivalent to roughly One-Nine) generates billions in revenue because it balances innovation with just enough reliability to meet customer expectations.

⚠ NOTE: The right reliability level is the one that meets customer expectations while preserving budget for innovation. More reliability than customers notice is waste. Less than they need is churn.

Interview Question 5: SRE vs DevOps — What is the difference?

SRE and DevOps are related but not identical:

Dimension	DevOps	SRE
Focus	Cultural movement; collaboration between Dev and Ops	Specific engineering discipline; implementation of DevOps principles
Nature	Philosophy and mindset	Concrete practices and tooling with measurable targets
Origin	Agile principles applied to delivery	DevOps principles applied to reliability at scale (Google)
Relationship	DevOps is the foundation SRE builds on	"SRE is an implementation of DevOps" — Ben Traynor Sloss
Role	No specific job role defined	SRE is a specific job role and specialization

⚠ NOTE: "SRE is what you get when you treat operations as a software engineering problem." DevOps is the philosophy; SRE is one concrete way to implement it, with specific practices, metrics, and roles.

Interview Question 6: What are SRE Team Formats?

In an interview, name all four formats and know the tradeoffs:

- **Dedicated SRE team (consultants)** — Central team guides others. Risk: perceived as external; less influence.
- **Ops team IS the SRE** — Existing ops adopts SRE practices. Risk: Dev and Ops remain separated.
- **SREs embedded in Dev teams** — True collaboration; high ownership. Risk: SREs become isolated from each other — hence SRE Guilds.
- **Hybrid (Dedicated + Embedded)** — Best coverage for large orgs. Risk: requires investment, rotation management, SRE career ladder.

⚠ NOTE: No team format is universally correct. Always say the right format depends on company size, service criticality, SRE maturity, and how open the teams are to SRE principles.

Quick Reference: Key SRE Terms

Term	Definition	Key Point
SLI	Service Level Indicator — measurement of reliability (good/bad event ratio)	How you measure if you're meeting targets
SLA	Service Level Agreement — contractual minimum reliability commitment	The floor, not the goal
SLO	Service Level Objective — internal desired reliability target	Above SLA, realistic, must give innovation margin
Error Budget	Allowed amount of unreliability derived from SLO	When exhausted, releases stop
Toil	Repetitive, manual, low-value, non-scaling work	Must be eliminated through automation
Blast Radius	The scope of customer impact from an incident	Minimize first; investigate later

On-Call	Designated engineer available to respond to incidents	Built on monitoring alerts
Postmortem	Formal document recording incident timeline and root cause	Must be blameless; purpose is learning
SREConf	Annual conference promoting SRE practices (USENIX)	Where SRE went from internal Google practice to public discipline
SRE Guild	Regular knowledge-sharing meetup for SREs across teams	Prevents isolation of embedded SREs
Five-Nines	99.999% availability = ~26 seconds downtime/month	Expensive; rarely the right target